

Cuestionario - Modo Protegido

1. Mecanismos de Protección de acceso al Segmento en Modo Protegido:
 - a. Realice un diagrama completo que permita comprender el mecanismo de protección de memoria del 80x386 que permite un no acceso directo al segmento. Explique detalladamente como interpreta un valor en un registro de segmento si TI=1 o TI=0.
 - b. ¿Cuáles son las condiciones que deben cumplirse para que el selector correspondiente pueda acceder al segmento? ¿Cuál es la relación entre RPL y DPL?
 - c. Explique que son y para qué sirven los siguientes registros y bits: GDTR, LDTR, bit G, bit D, bit P, bit S, bit E, bit A.
 - d. Práctica: Ejercicio de MP (Interpretación y funcionamiento) Consigna: Realice un diagrama interpretando en Modo Protegido el siguiente selector: CS=A3FF.
2. Descriptores de Segmento y Descriptores de Sistema:
 - a. ¿Cuántas tablas de descriptores existen en la arquitectura 80x386? Explique cada una de ellas. NOTA: No es lo mismo "tablas de descriptores" que "descriptores".
 - b. Explique de qué forma actúa el bit S como atributo de un descriptor. Utilice esta información para decir en qué tipo de descriptor se convierte él mismo. ¿Qué "tipos" de estos descriptores existen?
 - c. Explique el concepto y funcionamiento de un "Call via Gate".
3. Segmento de Estado de la Tarea (TSS):
 - a. ¿Qué se busca con el uso del TSS relacionado al uso del procesador y la conmutación de tareas?
 - b. Explique el funcionamiento y realice un diagrama de interpretación del mismo. Esquema del TSS.
4. Paginado:
 - a. Explique conceptualmente para qué sirve el proceso de paginado y cómo se relaciona con el sistema de segmentado.
 - b. ¿Qué hace la unidad de paginado? ¿Cómo lo hace?

Respuestas

1. Mecanismos de Protección de acceso al Segmento en Modo Protegido
 - a. Para el acceso a un segmento de memoria, por ejemplo en el 80386, se debe utilizar un selector en el cual la tarea a acceder cuente con el mismo nivel de privilegio o mayor al del segmento al que se pretende ingresar.
El selector cuenta con la siguiente información:

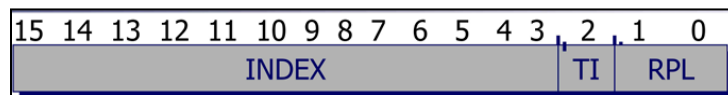


Figura 1. Selector de descriptor

- INDEX: 13 bits que seleccionan uno de los 8192 (13 bits) descriptores de la tabla seleccionada.
- TI: flag designando descriptor global (GDT) para 0 y descriptor local (LDT) para 1
- RPL: nivel de privilegio utilizado para solicitar el acceso a memoria. Siendo 0 el más alto y 3 el más bajo.

El selector ingresa a la tabla de descriptores, donde se selecciona el descriptor, compuesto por 8 bytes, donde se encuentra la dirección base del segmento que se busca acceder, luego se conforma la dirección física utilizando la base del descriptor obtenido y un desplazamiento, en el cual la base se enmascara los primeros 4 bits y multiplica x16 para obtener la dirección base y luego se suma el desplazamiento para obtener la dirección física, la cual va a ser cargada al bus de direcciones.

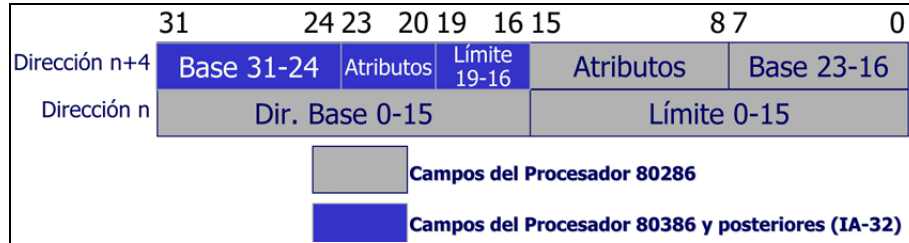


Figura 2. Contenido de un descriptor

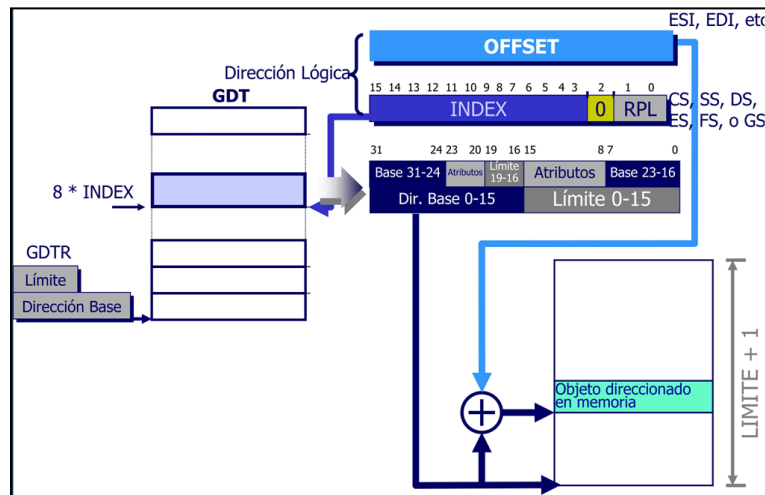


Figura 3. Acceso a segmento de memoria global

- b. El acceso a la memoria debe ser respetado según qué tarea solicite el acceso, ya que las tareas solo pueden obtener información de los segmentos locales a la tarea y los segmentos globales. Además, el offset seleccionado debe ser menor al límite del segmento y este debe estar presente, verificado a través del bit P del descriptor. Luego se verifican los privilegios, donde el nivel de privilegio actual de la tarea (CPL) y el nivel de privilegio del selector utilizado (RPL), debe ser menor o igual al nivel de privilegio del descriptor (DPL). Al terminar la validación se computa el nivel de privilegio efectivo (EPL), el cual es el nivel de privilegio menor entre el RPL y DPL.
- c. En modo protegido de la arquitectura 80x386 se cuenta con los siguientes conjuntos de registros:
- GDTR: es el registro donde se almacena la dirección base de 32 bits de la tabla global de descriptores.
 - GDTR: se encuentra dentro del GDTR y es el límite de la tabla global de descriptores, compuesto por 16 bits.
 - RPL: es el nivel de privilegio solicitado, se encuentra en el bit 0 y 1 del selector de descriptores, para comprobar los niveles de privilegio como se detallo en el ítem a y b.
 - LDTR: es el registro donde se almacena el selector que apunta al descriptor de la tabla local de descriptores que se esté utilizando en ese momento, este descriptor se encuentra dentro de la GDT.
 - FLAGS:
 - Bit G: Granularidad, bit 7 del byte 6, si es 0 el límite se define por los bits 19-0, si es 1 estos bits se corren 12 bits a la derecha, multiplicando por 4096.

- Bit D: Default, bit 6 del byte 6, si es 0 se utilizan segmentos de 16 bits y si es 1 se utilizan segmentos de 32 bits.
 - Bit P: Presente, bit 7 del byte 5, si es 0 el segmento no está en memoria y si es 1 existe en la memoria física. Si se intenta acceder con P=0 se obtiene un fallo de segmentación/pila (11 o 12).
 - Bit S: descriptor de segmento, bit 4 del byte 5, si es 0 es un descriptor de sistema y si es 1 es un descriptor de código o datos.
 - Bit E: Ejecutable, bit 3 del byte 5, si es 0 es segmento de datos y si es 1 es segmento de código.
 - Bit A: Accedido, bit 0 del byte 5, si es 0 el segmento no fue accedido y si es 1 se ha cargado el registro de segmento o utilizado por instrucciones de test de selectores.
- d. A partir del selector CS=A3FF se puede obtener la información del índice, TI y RPL de un segmento de código (CS).
- Índice: 101000111111 en binario y 1311 en decimal, planteando el acceso al descriptor 1311.
 - TI: 1, refiriendo a una tabla local de selectores.
 - RPL: 11, refiriendo al menor nivel de prioridad

Con esta información se sigue el siguiente flujo:

1. Tomando el LDTR se obtiene el selector de la LDT, se busca en la GDT, por medio del GDTR para obtener el LDT.
2. El índice 1311 se multiplica por 8 para obtener el segmento de código.
3. El segmento de código y la LDT obtienen el valor de base, límite y atributos
4. Con el valor de base y el offset se obtiene la dirección física en memoria.

2. Descriptores de Segmento y Descriptores de Sistema

- a. Existen 3 tipos de tablas de descriptores en la arquitectura 80x386, cada tabla es de longitud variable con un máximo de 64 KB y un máximo de 8192 descriptores.
- i. Tabla global de descriptores (GDT):
 - Descriptores disponibles para todas las tareas del sistema
 - Contiene segmentos de código y datos utilizados por el sistema operativo, los descriptores de Segmento de Estado de Tarea (TSS) y los descriptores para las Tablas de Descriptores Locales (LDT)
 - ii. Tabla de Descriptores Locales (LDT):
 - Contiene descriptores asociados con una tarea particular
 - Su propósito es aislar los segmentos de código y datos de una tarea dada del sistema operativo y de otras tareas
 - Puede contener descriptores de código, datos, pila, compuertas de tarea y compuertas de llamada
 - iii. Tabla de Descriptores de Interrupción (IDT):
 - Contiene descriptores que apuntan a la ubicación de hasta 256 rutinas de servicio de interrupción
 - En modo protegido, a diferencia del modo real, no almacena vectores de interrupción, sino descriptores.
 - Su dirección base y límite se almacenan en el registro IDTR
- b. El bit S, como se detallo en el punto 1.c, clasifica si el descriptor el comportamiento del descriptor.
- Si S=1, se comporta como un descriptor de segmento de código o datos, donde se toma el bit E para clasificar entre:
- Código (E=1) donde depende el acceso del bit Conforme (C) y el bit de lectura habilitada (R).

- Datos (E=0) donde depende del bit de escritura (W) y la dirección de expansión (ED) donde ED=0 el segmento se expande hacia arriba y ED=1 se expande hacia abajo. Si S=0, se comporta como un descriptor de sistema, utilizándose para estructuras internas del sistema. El descriptor está codificado en los 4 bits menos significativos:

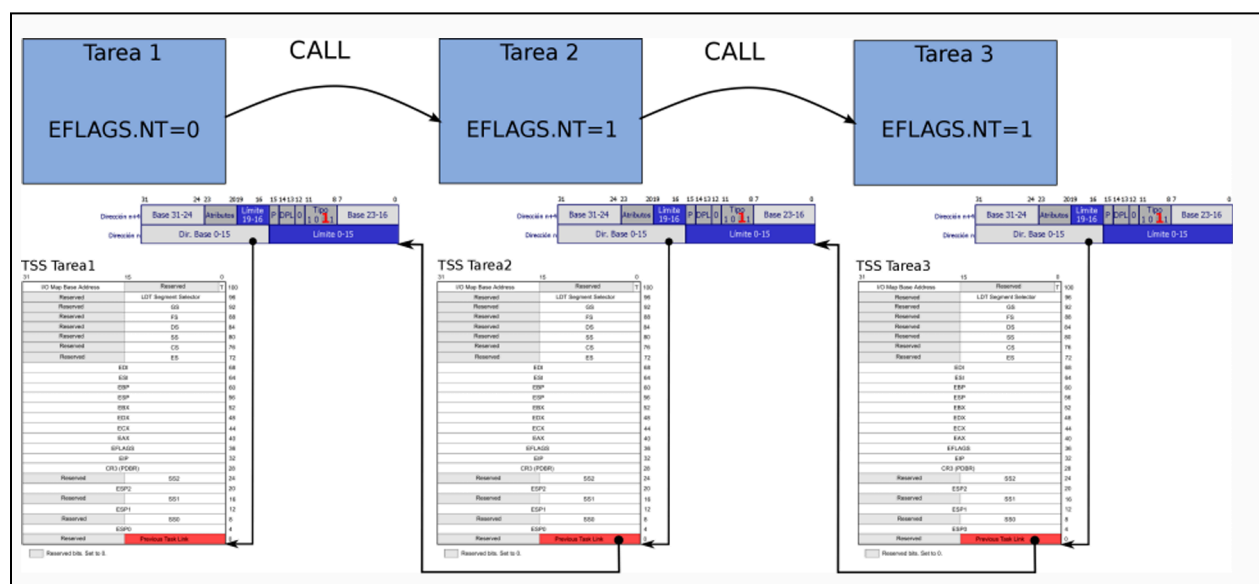
0000: Inválido	1000: Inválido
0001: TSS tipo 286 disponible	1001: TSS tipo 386 disponible
0010: LDT	1010: Inválido
0011: TSS tipo 286 ocupado	1011: TSS tipo 386 ocupado
0100: Compuerta de llamada tipo 286	1100: Compuerta de llamada tipo 386
0101: Compuerta de tarea tipo 286 ó 386	1101: Inválido
0110: Compuerta de interrupción tipo 286	1110: Compuerta de interrupción tipo 286
0111: Compuerta de trampa tipo 286	1111: Compuerta de trampa tipo 286

Figura 4. Descriptores del sistema

- c. Un "Call via Gate" permite que una tarea con menor nivel de privilegio pueda acceder a rutinas de mayor nivel, modificando momentáneamente los niveles de prioridad para permitirlo. Este tipo de mecanismo es esencial en sistemas operativos modernos que utilizan protección de memoria porque permite que aplicaciones de usuario usen función del Kernel y system calls de una forma que puedan ser controladas por el sistema operativo.
Las "Call Gates" usan un selector de valor especial para referenciar a un descriptor accedido por el GDT o el LDT, que contiene la información necesaria para la llamada a través de niveles de privilegio. Esto es similar al mecanismo usado por las interrupciones. Si esto se realiza a través de un CALL indica que la rutina llamada deberá retornar a la tarea que la ejecuta, por lo que se producirá un anidamiento de tareas (nest).
3. Segmento de Estado de la Tarea (TSS)
 - a. El concepto de TSS permite la ejecución de múltiples tareas en "simultáneo", simulando múltiples procesadores. Utilizando el segmento de estado de tarea permite que el procesador conmute entre tareas con velocidad, guardando el estado actual de cada tarea en ejecución, con una velocidad alta, suficiente para que el usuario perciba que se ejecutan en simultáneo. El orden de ejecución de estas tareas y los cambios de contexto por medio de los TSS están definidos por las políticas de scheduling del sistema operativo utilizado.
 - b. El procesador utiliza el Registro de Tarea (TR), que contiene un selector que apunta al descriptor del TSS de la tarea actualmente en ejecución. El TSS contiene la información completa sobre el estado de una tarea y un campo de enlace para tareas anidadas.
Cuando el scheduler decide que la tarea va a ser conmutada, además de guardar los valores de los registros generales (EAX, EBX, etc.) e indicadores (EFLAGS), el TSS es crucial para la gestión de pilas cuando se cruzan niveles de privilegio, ya que almacena los valores de SS y ESP para los stacks de nivel 0, 1 y 2. Luego, procede a cargar el TSS de la tarea a ejecutar, hasta que se seleccione realizar una conmutación y realiza el proceso de guardado y carga de TSS nuevamente.

31	15	0
I/O Map Base Address	Reserved	T
Reserved	LDT Segment Selector	
Reserved	GS	
Reserved	FS	
Reserved	DS	
Reserved	SS	
Reserved	CS	
Reserved	ES	
EDI		
ESI		
EBP		
ESP		
EBX		
EDX		
ECX		
EAX		
EFLAGS		
EIP		
CR3 (PDBR)		
Reserved	SS2	
ESP2		
Reserved	SS1	
ESP1		
Reserved	SS0	
ESP0		
Reserved	Previous Task Link	

Figura 5. Registro de TSS#



4. Paginado

- a. El proceso de paginado sirve para gestionar la memoria dividiendo los programas y el espacio de direcciones lineales en páginas, bloques de tamaño uniforme y fijo. Esto permite una gestión de memoria virtual, donde los espacios en memoria se encuentran segmentados. La incorporación de la paginación permite aumentar los espacios de memoria asignados a cada recurso del sistema, como que un programa pueda ser mayor que la memoria accesible de forma física, cargando cada página por separado.
La unidad de manejo de memoria (MMU) consiste en una unidad de segmentación (similar a la del 80286) y una unidad de paginado.
 - La segmentación permite el manejo del espacio de direcciones lógicas agregando un componente de direccionamiento extra, que permite que el código y los datos se puedan reubicar fácilmente.
 - El mecanismo de paginado opera por debajo y es transparente al proceso de segmentación, para permitir el manejo del espacio de direcciones físicas. Y si una página determinada no se encuentra en memoria, el 80386 se lo indica al sistema operativo mediante la excepción 14, luego éste carga dicha página desde el disco y finalmente puede seguir ejecutando el programa, como si hubiera estado dicha página todo el tiempo.
- b. La unidad de paginado se ubica entre la memoria física y la unidad de segmentación. Donde la dirección lineal producida por la unidad de segmentación se divide en tres partes, directorio (bits 31-22), tabla (bits 21-12) y offset (bits 11-0), ocupando los 2 primeros 4 KB cada uno.
El registro CR3 contiene la dirección física inicial del directorio de páginas, de la cual se selecciona el directorio objetivo, donde contiene la dirección inicial de la tabla de páginas, donde se aplica el desplazamiento para obtener la dirección inicial de la página objetivo. Con esta última y los 12 bits de offset se obtiene la dirección lineal.